

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки
09.03.01 Информатика и вычислительная техника

Направленность (профиль) «Технологии разработки программного
обеспечения и обработки больших данных»

Курсовая работа
по дисциплине «Прикладные информационные технологии»

«Развёртывание микросервисной архитектуры с использованием Docker
Compose и защищённого доступа через Let's Encrypt для приложения по
предсказанию одобрения ипотеки»

Обучающегося 3 курса
очной формы обучения
Гасяк А.К.

Руководитель курсовой работы:
старший преподаватель кафедры ИТЭО
Аксютин Павел Александрович

Санкт-Петербург
2026

Оглавление

ВВЕДЕНИЕ	3
1. Изучение Docker и микросервисной архитектуры	3
1.1 Понятие о Docker.....	3
1.2 Основные компоненты Docker.....	4
1.3 Микросервисная архитектура.....	8
1.4 Базы данных микросервисов	8
1.5 Сетевые вызовы.....	9
1.6 Принципы микросервисной архитектуры.....	10
1.7 Обзор и сравнение технологий	13
1.8 Системные требования.....	16
2. Реализация поставленной задачи	18
2.1 Dockerfile сервисов.....	18
2.2 Docker-compose.yaml.....	21
ЗАКЛЮЧЕНИЕ	28
СПИСОК ЛИТЕРАТУРЫ.....	28

ВВЕДЕНИЕ

В этой работе мы постараемся развернуть микросервисную архитектуру с использованием Docker Compose и защищённого доступа через Let's Encrypt для приложения по предсказанию одобрения ипотеки. Для этого мы изучим и повторим материалы по работе с Docker и Docker Compose. Вспомним, что такое Nginx-proxy и обеспечим автоматический выпуск сертификатов. Изучим альтернативные способы получения сертификатов и контейнеризации, чтобы выявить наиболее эффективный и подходящий инструмент.

Цель работы: разработать своё приложение, которое будет использовать несколько контейнеров, запаковать в образ(ы), написать для него собственный Docker Compose, настроить NGINX-Proxy и добавить автоматический выпуск сертификата.

Задачи исследования:

- 1) Обучиться работе с Docker Compose и образами Docker.
- 2) Изучить технологии и сервисы, которые используются в работе.
- 3) Освоить использование NGINX-Proxy и автоматический выпуск сертификатов

1. Изучение Docker и микросервисной архитектуры

1.1 Понятие о Docker

Docker – это инструмент контейнеризации, который позволяет собирать приложения вместе со всеми библиотеками, настройками и зависимостями в единый контейнер, который можно запустить в любой системе.

Docker используется, при переносе и работе с приложением на различных устройствах, он гарантирует, что приложение будет запускаться и работать везде одинаково независимо от систем и настроек.

Контейнеры также используют общее ядро операционной системы, вместо создания отдельных копий ОС, чтобы занимать меньше места и быстрее запускаться. Также каждый контейнер работает независимо от других контейнеров, поэтому если в одном из контейнеров произойдет сбой или утечка памяти, это не затронет другие контейнеры и систему в целом.

Основные способы использования Docker:

- Автоматизация упаковки и развертывания приложений.
- Создание собственных легковесных PaaS окружений.
- Автоматизация тестирования и непрерывной интеграции/развертывания.
- Развертывание и масштабирование веб-приложений, баз данных и сервисов бекенда.

1.2 Основные компоненты Docker

Компоненты Docker это ключевые элементы, из которых состоит платформа контейнеризации. Без них Docker не смог бы работать и выполнять свои функции. Его составляющие – работают вместе, позволяя упаковывать приложения со всеми зависимостями в изолированные среды, запускать их и управлять ими.

У Docker есть несколько основных компонентов:

- Docker Compose
- Docker Registry
- Docker Desktop
- Docker Host
- Docker Daemon
- Docker Client
- Docker Image

- Docker Container
- Dockerfile

Docker Compose

Docker Compose – это инструмент Docker, позволяющий запускать множество приложений, состоящих из нескольких Docker-контейнеров.

В отличие от Docker, который применяется для управления отдельными сервисами, из которых состоит приложение, Docker Compose используется для управления сразу несколькими серверами одного приложения одновременно.

Docker Registry

Docker Registry или реестры Docker, используются чтобы хранить созданные образы. Это хранилище позволяет загружать образы в реестр и извлекать их при необходимости. Существует два типа реестров: публичные и приватные. На публичных есть множество образов, таких как nginx, postgres, redis и другие официальные издания, обычно они общедоступные и используются множеством пользователей. Приватные – имеют запатентованные реестры и чаще всего используются крупными провайдерами.

Docker Desktop

Docker Desktop – это приложение для запуска и управления контейнерами Docker, позволяющее отображать графический интерфейс и CLI-команды, для локальной разработки. Он позволяет каждому контейнеру сохранять свою самодостаточность, оставляя в каждом контейнере нужные библиотеки и зависимости, чтобы избежать конфликтующих позиций между другими проектами.

Docker Host

Docker Host – это виртуальная машина или сервер, на котором установлен Docker и запускается Docker Daemon. Он хранит образы, собранные локально или загруженные из реестра, запускает контейнеры, являющиеся экземплярами образов, создаёт тома для постоянного хранения данных и настраивает управление сетями для связи контейнеров между собой.

Docker Daemon

Docker Daemon – это фоновый процесс-сервер, который управляет всеми Docker-объектами и исполняет все необходимые операции по работе с контейнерами. Например: Слушает запросы от Docker Client, строит образы из Dockerfile, запускает, останавливает и перезапускает контейнеры, управляет сетями и томами, скачивает образы из реестров, отвечает на запросы состояний.

Docker Client

Docker Client – это интерфейс, через который пользователь отдаёт команды Docker. Клиент принимает введенную пользователем команду, преобразует её в HTTP-запрос и отправляет в Docker Daemon через локальный сокет, в том случае, если клиент и Docker Daemon находятся на одной виртуальной машине, в ином случае, отправление происходит через TCP-соединение.

Docker Image

Docker Image – это централизованное место для хранения и обмена изображениями контейнеров. Их ещё называют «шаблонами» для создания контейнеров. Он сохраняет в себе карту среды с нужными для работы приложения составляющими, такими как: операционная система, библиотеки, зависимости, исходный код, переменные окружения и настройки.

Docker Container

Docker Container – помогает создавать и развёртывать контейнеры, внутри которых можно будет запускать приложения и сервисы.

Когда Docker запускает контейнер, место для записи информации остаётся пустым, но, когда происходят изменения оно заполняется записями – создаётся резервная копия недоступная для редактирования, но используемая для просмотра.

Dockerfile

Dockerfile содержит набор инструкций с аргументами. Каждая инструкция пишется заглавными буквами (например FROM). Инструкции обрабатываются сверху вниз. Каждая инструкция добавляет новый слой в образ и коммитит изменения. Docker исполняет инструкции, следуя процессу:

1. Запустить контейнера из образа
2. Исполнить инструкции и внести изменения в контейнер
3. Запустить эквивалент `docker commit` для записи изменений в новый слой образа
4. Запустить новый контейнер из нового образа
5. Исполнить следующую инструкцию в файле и повторить весь процесс сначала.

1.3 Микросервисная архитектура

Микросервисная архитектура – это организация распределённой системы на основе микросервисов и их взаимодействия друг с другом и со средой по сети, выстроенная по определённым принципам, правилам направляющим проектирование архитектуры.

Микросервис – это конкретное приложение, которое выполняет одну определённую функцию. Это самостоятельная программа, у которой есть свой код, своя база данных, свои зависимости и свой процесс. Он является частью микросервисной архитектуры.

В отличии от Монолитной архитектуры, которая является одним большим приложением, где весь код лежит в одном месте, все функции работают в одном процессе, а все данные хранятся в одной базе данных, микросервисная архитектура – поделена на много маленьких приложений, и может развёртывать каждый сервис отдельно, что позволяет эффективнее разрабатывать проекты и быстрее с меньшим количеством потерь решать возникающие проблемы с сервисами. В случае монолита при возникновении ошибки, в любой части кода произойдёт падение всего приложения. В микросервисной архитектуре при появлении ошибки в одном из сервисов влияние не будет распространяться на остальные сервисы.

1.4 Базы данных микросервисов

Каждый микросервис использует свою базу данных для хранения данных. Связано это с тем, что данные на разных микросервисах могут отличаться и в случае разобщения пострадать могут другие микросервисы. Разобщение в том числе могло бы возникнуть из-за возможности сервисов напрямую изменять данные в своём хранилище данных и в случае, если бы хранилище было общим, это могло бы изменить структуру таблиц у других микросервисов.

Также разные базы данных для серверов помогают избежать проблем конкуренции. Каждый сервис пытался бы быстрее остальных соединиться с базой данных, пытаясь выполнить свой процесс, затрачивая на это больше оперативной памяти чем необходимо и больше времени. Это бы заметно усложнило и замедлило процесс, так как в одной базе данных разные микросервисы пытались бы выполнять разные операции.

В тех случаях, когда на один из сервисов производится большая нагрузка, можно запустить несколько экземпляров одного и того же сервиса, чтобы увеличить его производительность. При этом это не будет влиять на производительность других сервисов, что позволит уменьшить нагрузку и избежать замедления процессов.

Таким образом разные базы данных позволяют каждому сервису обновлять свою базу данных независимо от других. Абстрагироваться от ошибок других сервисов в том случае, когда база данных одного из сервисов может упасть. Использовать тот тип базы данных, который будет лучше подходить для задач сервиса. Обеспечить безопасность на уровне данных и настроить разные уровни доступа для каждого сервиса отдельно.

1.5 Сетевые вызовы

Микросервисы также способны взаимодействовать друг с другом через сетевые вызовы используя для этого HTTP-запросы.

Происходит это следующим образом, сервис отправляет post-запрос на URL другого сервиса, внутри сети Docker находит соответствующий контейнер, и через открытый для сервиса порт передаёт ему запрос, сервис обрабатывает запрос и возвращает ответ.

Всего существует два типа сетевых вызовов: синхронный и асинхронный.

Синхронный вызов наиболее распространён и часто используется. Он заключается в таком взаимодействии между сервисами, при котором один сервис посылает запрос другому сервису и приостанавливает свою работу, ожидая ответа. Когда запрос будет обработан, а ответ – возвращён, сервис вновь продолжит свою работу. Такой обмен запросами легко реализуем и даёт быстрый результат, однако это также останавливает работу одного из сервисов до тех пор, пока вызов не завершится.

Асинхронный вызов – способ при котором сервис отправляет запрос и не дожидаясь ответа продолжает свою работу. В отличии от синхронного, временной остановки сервиса не происходит, он продолжает работать и параллельно ожидает ответа. Это позволяет повысить общую производительность и отказоустойчивость системы, но также это не гарантирует, что запрос будет обработан, и поэтому требуется предельная внимательность при отладке.

1.6 Принципы микросервисной архитектуры

Микросервисная архитектура считается довольно гибкой в использовании, во многом благодаря своим принципам и правилам. Это является её главным отличием от монолита и это помогает ей оставаться отказоустойчивой и масштабируемой.

Некоторые из архитектурных принципов:

- Принцип 1. Моделирование вокруг бизнес-задач.
- Принцип 2. Независимое развёртывание.
- Принцип 3. Децентрализованное управление данными.
- Принцип 4. Общение через API.
- Принцип 5. Устойчивость к ошибкам.
- Принцип 6. Обнаружение сервисов.
- Принцип 7. Независимое масштабирование.

- Принцип 8. Автоматическое развёртывание (CI/CD).
- Принцип 9. Разработка в разных командах.
- Принцип 10. Инфраструктура как код.

Принцип 1. Каждый микросервис должен решать одну конкретную бизнес-задачу. Этот принцип гласит, что разбиение системы должно происходить по бизнес-функциям, а не по техническим слоям (как в монолите). Нужно чётко определить, какую бизнес-задачу будет решать этот конкретный сервер

Принцип 2. Каждый микросервис должен развертываться, обновляться и масштабироваться независимо от других сервисов. Этот принцип гласит, что можно обновлять один сервис и при этом не перезапускать другой, для продолжения работы.

Принцип 3. Каждый микросервис управляет своей собственной базой данных. Другие сервисы не имеют доступа к этой БД напрямую, только через API. Ранее мы разбирали базы данных микросервисов и причины, из-за которых каждый микросервис использует свою базу данных.

Принцип 4. Сервисы общаются друг с другом только через четко определенные API (интерфейсы). Внутренняя реализация сервиса скрыта. Это означает, что сервисы должны получать информацию о том, что делает другой сервис, но не обязаны получать информацию о способе выполнения действий.

Принцип 5. Система должна продолжать работать даже при сбоях в отдельных сервисах. Это означает, что сервисы должны продолжать работать не зависимо друг от друга, и что ошибка в одном сервисе не должна мешать работе других сервисов.

Принцип 6. Сервисы должны находить друг друга автоматически. Это означает, что сервисы должны уметь обращаться друг к другу для дальнейшего взаимодействия. Для этого можно использовать встроенный в Docker DNS-сервер и тогда имена серверов будут преобразованы в IP-адреса контейнеров.

Принцип 7. Каждый микросервис можно масштабировать независимо от других. Ранее мы также разбирали этот принцип на примере большой нагрузки на один из серверов и способа решения этой проблемы через запуск нескольких экземпляров одного и того же сервиса. Это не должно повлиять на работу других сервисов.

Принцип 8. Развертывание микросервисов должно быть полностью автоматизированным. Это означает, что при запуске кода в репозиторий, так же, как и сборка Docker-образа будет запускаться автоматически и отправляться в реестр, после чего сервис также автоматически обновиться.

Принцип 9. Разные микросервисы могут разрабатываться разными командами, которые работают независимо. В данном принципе рассматривается распределение задач по командам и разработка каждого микросервиса отдельной командой. Однако независимо от используемой технологии и кода итоговый результат должен коррелироваться с остальными микросервисами.

Принцип 10. Вся инфраструктура (сервисы, сети, volumes) описывается в коде и хранится в репозитории. Этот принцип помогает легко поднять окружение на новом сервере, вернуться к предыдущей версии в случае ошибок или недоработок.

1.7 Обзор и сравнение технологий

В рамках проекта используется микросервисная архитектура, развернутая с помощью Docker и Docker Compose. Каждый сервис изолирован в отдельном контейнере, что обеспечивает независимость разработки и развертывания.

1. Docker

Назначение: Контейнеризация приложений

Роль в проекте: Упаковка каждого сервиса в отдельный контейнер

Альтернативы: VirtualBox, VMware

Обоснование выбора:

Таблица 1. – Сравнение с Docker

Критерий	Docker	VirtualBox	VMware
Уровень изоляции	Процессная	Полная	Полная
Ресурсоёмкость	Низкая	Высокая	Высокая
Скорость запуска	Высокая (несколько секунд)	Средняя (несколько минут)	Средняя (несколько минут)
Портативность	Высокая (образ – файл)	Средняя (зависит от гипервизора)	Средняя (зависит от гипервизора)

Вывод: Таким образом Docker оказался самым менее ресурсозатратным, имеющим высокую скорость запуска и высокую скорость развёртывания.

2. Docker Compose

Назначение: Управление несколькими контейнерами

Роль в проекте: Обеспечение запуска всех сервисов одной командой

Альтернативы: Kubernetes, Swarm

Обоснование выбора:

Таблица 2 – Сравнение с Docker Compose

Критерий	Docker Compose	Swarm	Kubernetes
Сложность использования	Низкая	Средняя	Высокая
Отказоустойчивость	Средняя	Средняя	Высокая
Масштабирование	Ручное	Ручное	Автоматическое
Сетевые возможности	Простая сеть внутри одного хоста	Виртуальная сеть поверх базовой	Работа через получение IP-адресов для контейнеров

Вывод: Таким образом Docker Compose, используется благодаря своей простоте использования и освоения.

3. NGINX-Proxy

Назначение: Автоматическое ретранслирование запросов на сервера

Роль в проекте: Маршрутизация запросов к сервисам и поддержка HTTPS

Альтернативы: Traefik, Caddy

Обоснование выбора:

Таблица 3 – Сравнение с Nginx-проху

Критерий	Nginx-проху	Traefik	Caddy
Конфигурация	Ручная	Динамическая	Лаконичная
Производительность	Очень высокая	Хорошая	Хорошая
Порог входа	Средний	Средний	Низкий
Расширяемость	Через модули	Через middlewares	Через плагины
Настройка	Автоматическая	Автоматическая	Ручная
Обновление	Автоматическое	Требует перезагрузки	Требует перезагрузки

Вывод: Таким образом Nginx-проху был выбран благодаря своей высокой производительности и автоматизации, упрощающей поддержку системы.

4. Acme-companion

Назначение: Автоматическое получение SSL-сертификатов

Роль в проекте: Обеспечение HTTPS протокола для домена smth.ru

Альтернативы: Certbot

Обоснование выбора:

Таблица 4 – Сравнение с acme-companion

Критерий	Acme-companion	Certbot
Интеграция с инфраструктурой	Тесно интегрирован с Docker	Интегрируется с веб-сервером
Управление конфигурацией	Работает вместе с nginx-проху	Работает вместе с плагинами
Сложность использования	Низкая	Средняя
Удобство в CI/CD	Высокое	Среднее

Вывод: Таким образом Acme-companion был выбран из-за лучшей интеграции с Docker и связки с nginx-проху.

5. Микросервисная архитектура

Назначение: Построение приложения

Роль в проекте: Обеспечение корректной работы всех модулей приложения

Альтернативы: Монолитная архитектура (Монолит)

Обоснование выбора:

Критерий	Микросервисная архитектура	Монолитная архитектура
Структура приложения	Набор небольших независимых сервисов	Единое приложение, все компоненты собраны в один деплой
Масштабирование	Возможность масштабировать отдельные сервисы	Масштабируется целиком
Отказоустойчивость	Высокая	Низкая
Производительность	Высокая	Низкая
Сложность использования	Средняя	Низкая

Вывод: Таким образом микросервисная архитектура лучше подходит для реализации приложения для предсказания одобрения кредита, так как микросервисы имеют более высокую производительность и высокую отказоустойчивость.

1.8 Системные требования

Требования к компьютеру:

- CPU: 2 – 4 ядра
- RAM: 4 Гб
- Диск: 10 Гб
- Сеть: 10 Мбит/с

Сетевые требования:

- Доступ в интернет
- Свободные порты: 8080, 8081, 8082, 80, 443

Операционная система:

- Linux: Ubuntu 20.04 – 22.04
- Windows: Windows 10 – 11
- macOS: macOS 11 – 13

База данных:

- SQLite: 3.x

Языки программирования:

- Python: 3.9+
- JavaScript: ES2023+

Фреймворки и библиотеки:

- FastAPI: 0.100+ (REST API)
- Uvicorn: 0.23+ (ASGI-сервер)
- scikit-learn: 1.3+ (ML)
- pandas: 2.0+ (обработка данных)
- python-multipart: 0.0.6+ (обработка загруженных файлов)
- numpy: 1.24+ (численные вычисления)
- joblib: 1.3+ (сохранение моделей)
- SQLAlchemy: 2.0+ (ORM)

- aiosqlite: 0.19+ (асинхронный SQLite)

Таким образом мы установили необходимые системные требования для работы приложения.

2. Реализация поставленной задачи

2.1 Dockerfile сервисов

Приложение 1. Программный код Dockerfile для storage-service:

```
FROM python:3.13-slim AS builder

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    g++ \
    make \
    python3-dev \
    libffi-dev \
    libssl-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip wheel setuptools && \
    pip wheel --no-cache-dir --wheel-dir=/wheels -r requirements.txt

FROM python:3.13-slim

WORKDIR /app

COPY --from=builder /wheels /wheels
COPY --from=builder /app/requirements.txt .

RUN pip install --no-cache-dir --find-links=/wheels -r requirements.txt && \
    rm -rf /wheels

COPY . .

# fallback
ENV STORAGE_SERVICE_PORT=8081
EXPOSE ${STORAGE_SERVICE_PORT}

CMD uvicorn src:app --reload --host 0.0.0.0 --port $STORAGE_SERVICE_PORT
```

Приложение 2. Программный код Dockerfile для trainer-service:

```
FROM python:3.13-slim AS builder

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    g++ \
    make \
    python3-dev \
    libffi-dev \
    libssl-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip wheel setuptools && \
    pip wheel --no-cache-dir --wheel-dir=/wheels -r requirements.txt

FROM python:3.13-slim

WORKDIR /app

COPY --from=builder /wheels /wheels
COPY --from=builder /app/requirements.txt .

RUN pip install --no-cache-dir --find-links=/wheels -r requirements.txt && \
    rm -rf /wheels

COPY . .

# fallback
ENV TRAINER_SERVICE_PORT=8082
EXPOSE ${TRAINER_SERVICE_PORT}

CMD uvicorn src:app --reload --host 0.0.0.0 --port $TRAINER_SERVICE_PORT
```

Приложение 3. Программный код Dockerfile для client-service:

```
FROM python:3.13-slim AS builder

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    g++ \
    make \
    python3-dev \
    libffi-dev \
    libssl-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip wheel setuptools && \
    pip wheel --no-cache-dir --wheel-dir=/wheels -r requirements.txt

FROM python:3.13-slim

WORKDIR /app

COPY --from=builder /wheels /wheels
COPY --from=builder /app/requirements.txt .

RUN pip install --no-cache-dir --find-links=/wheels -r requirements.txt && \
    rm -rf /wheels

COPY . .

# fallback
ENV CLIENT_SERVICE_PORT=8080
EXPOSE ${CLIENT_SERVICE_PORT}

CMD uvicorn src:app --reload --host 0.0.0.0 --port $CLIENT_SERVICE_PORT
```

2.2 Docker-compose.yaml

Приложение 4. Программный код Docker-compose.yaml:

```
services:
  nginx-proxy:
    image: nginxproxy/nginx-proxy:latest
    container_name: nginx-proxy
    ports:
      - "80:80"
      - "443:443"
```

Рис.1 – «nginx-проху» – обратный прокси-сервер

Здесь прослушиваются порты 80 и 443, автоматически маршрутизируются запросы к контейнерам на основе переменной VIRTUAL_HOST. Происходит поддержание Let's Encrypt для HTTPS.

```
volumes:
  - /var/run/docker.sock:/tmp/docker.sock:ro
  - certs:/etc/nginx/certs:ro
  - vhost:/etc/nginx/vhost.d
  - html:/usr/share/nginx/html
```

Рис.2 – Постоянное хранение данных

Здесь происходит монтирование папок, для того, чтобы nginx-проху мог взаимодействовать с другими контейнерами и автоматически хранить, и обновлять конфигурацию не зависимо от контейнеров, даже при их удалении, файлы всё равно будут сохраняться.

```
acme-companion:
  image: nginxproxy/acme-companion:latest
  container_name: nginx-proxy-acme
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock:ro
    - certs:/etc/nginx/certs:rw
    - vhost:/etc/nginx/vhost.d:rw
    - html:/usr/share/nginx/html:rw
    - acme:/etc/acme.sh
  environment:
    - DEFAULT_EMAIL=domentemp@opemails.com
  depends_on:
    - nginx-proxy
  restart: unless-stopped
```

Рис. 3 – «acme-companion» – автоматическое получение SSL-сертификатов

Здесь происходит работа с nginx-проху, автоматически приобретаются SSL-сертификаты от Let's Encrypt и обновляются по истечению своего срока. Контейнер с VIRTUAL_HOST=smth.ru и LETSENCRYPT_HOST=smth.ru запускается, асme-companion реагирует на контейнер и запрашивает сертификат. Сертификат сохраняется в «volumes: certs», после чего nginx-проху автоматически начинает использовать HTTPS.

```
client:
  image: nginx
  volumes:
    - templates:/etc/nginx/templates
  expose:
    - "80"
  environment:
    - VIRTUAL_HOST=smth.ru
    - LETSENCRYPT_HOST=smth.ru
  restart: always
```

Рис. 4 – Фронтенд

Здесь происходит передача запроса на развёртывание шаблона пользовательской среды и выявления визуальной составляющей микросервисного приложения. Проксируются запросы на smth.ru через nginx-проху и автоматически происходит получение SSL-сертификата.

```
client-service:
  build:
    context: ./client-service
    dockerfile: Dockerfile
  container_name: client-service
  ports:
    - "8080:8080"
  environment:
    - CLIENT_SERVICE_PORT=8080
    - STORAGE_SERVICE_API=http://storage-service:8081
    - TRAINER_SERVICE_API=http://predictor-service:8082
    - PREDICTOR_SERVICE_API=http://predictor-service:8083
  depends_on:
    - storage-service
    - trainer-service
  networks:
    - app-network
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 10s
  restart: unless-stopped
```

Рис. 5 – Основной бэкенд-сервис

Здесь обрабатываются клиентские запросы и перераспределяются между сервисом для хранения файлов, сервисом обучения модели и сервисом для прогнозирования предсказаний. Перед запуском, проверяется, запущены ли сервисы для хранения данных (storage-service) и обучения модели (trainer-service), а также Docker проверяет, жив ли сервис через эндпоинт «/health» здесь и в дальнейшем эта процедура будет проверять и другие сервисы.

```
storage-service:
  build:
    context: ./storage-service
    dockerfile: Dockerfile
  container_name: storage-service
  ports:
    - "8081:8081"
  environment:
    - UPLOAD_DIR=/uploads
    - SQLITE_URL=sqlite+aiosqlite:///db/storedfiles.db
    - STORAGE_SERVICE_PORT=8081
  volumes:
    - uploads:/uploads
    - db:/db
  networks:
    - app-network
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8081/health"]
    interval: 300s
    timeout: 10s
    retries: 3
    start_period: 10s
  restart: unless-stopped
```

Рис. 6 – «storage-service» – сервис для хранения файлов

Здесь происходит сохранение загруженных файлов и их хранение с помощью встроенной системы управления базами данных SQLite. Таким образом клиент загружает CSV-файл через client-service, client-service сохраняет его в storage-service. Файл сохраняется в /uploads, а информация о файле записывается в SQLite.


```

trainer-service:
  build:
    context: ./trainer-service
    dockerfile: Dockerfile
  container_name: trainer-service
  ports:
    - "8082:8082"
  environment:
    - TRAINER_SERVICE_PORT=8082
    - STORAGE_SERVICE_API=http://storage-service:8081
  depends_on:
    - storage-service
  networks:
    - app-network
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8082/health"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 10s
  restart: unless-stopped

```

Рис. 7 – «trainer-service» – обучение моделей

Здесь происходит обучение модели машинного обучения и использования её для предсказания одобрения кредита. Для этого данные загружаются из «storage-service» и на их основе модель начинает обучаться. Обученные модели также сохраняются в storage.

Приложение 5. Раздел «volumes»

```

volumes:
  templates: # Для фроненда
  uploads:   # Загруженные файлы
  db:        # База данных SQLite
  certs:     # SSL-сертификаты
  vhost:     # Конфиги nginx
  html:      # Статика для Let's Encrypt
  acme:      # Данные acme-companion

```

Приложение 6. Раздел «networks»

```
networks:  
  app-network:  
    driver: bridge
```

Приложение 7. Ключевые понятия:

Термин	Объяснение
build:	Docker собирает образ из Dockerfile в указанной папке
image:	Использует готовый образ из Docker Hub
ports:	Пробрасывает порты из контейнера на хост
expose:	Открывает порт внутри сети Docker (не пробрасывает наружу)
volumes:	Монтирует папки для постоянного хранения данных
environment:	Переменные окружения для контейнера
depends_on:	Запускает сервисы в правильном порядке
healthcheck:	Проверяет, работает ли сервис
networks:	Создает виртуальную сеть для общения контейнеров

Приложение 8. Итоговый вид приложения:

ML Model Service API

Выбор модели для предсказания

Выбрать	UID	Имя файла
☐	a1b2c3d4-e5f6-7890-abcd-e1234567890	credit_model_v1.pkl
☒	b2c3d4e5-f6a7-8901-bcde-f12345678901	credit_model_v2.pkl

Ввод признаков клиента

№	Возраст	Пол	Образование	Доход	Стаж (мес)	Тип жилья	Сумма кредита	Цель кредита	Ставка %	Доля дохода	Ист. кредита (мес)	Кред. рейтинг	Просрочки
1	30	Женский	Магистр	50000	24	Собственность	100000	Образование	12,5	0,25	36	700	Нет

[\[+\] Добавить запись](#)

Действия с предсказаниями

[Сделать предсказание](#)[Загрузить CSV](#)

Модель выбрана. Можно выполнять предсказания.

Ответ от модели:
Модель: credit_model_v2.pkl
UID: model_v2
Источник: Табличный ввод

Активация Windows
Чтобы активировать Windows, перейдите в раздел "Параметры".

Действия с предсказаниями

[Сделать предсказание](#) [Загрузить CSV](#)

Модель выбрана. Можно выполнять предсказания.

Ответ от модели:

Модель: credit_model_v2.pkl
UID: model_v2
Источник: Табличный ввод
Количество записей: 1

Клиент #1:

Возраст: 30
Пол: Женский
Образование: Master
Доход: 50 000 P
Стаж: 24 мес.
Жилье: Own
Сумма кредита: 100 000 P
Цель: EDUCATION
Ставка: 12.5%
Доля дохода: 25.0%
Ист. кредита: 36 мес.
Кред. рейтинг: 700
Просрочки: Нет
Скоринг: 86%
Решение: ОДОБРЕНО (вероятность: 0.86)

Итоговая статистика:

Одобрено: 1 из 1 (100.0%)
Отказано: 0 из 1 (0.0%)

ЗАКЛЮЧЕНИЕ

Таким образом, мы развернули микросервисную архитектуру с использованием Docker Compose. Мы также обеспечили ей защищённый доступ через Let's Encrypt, чтобы она могла выполнять свои функции, как приложение для предсказания одобрения ипотеки. Выбранные технологии Docker, Docker Compose, nginx-проxy, acme-companion помогли обеспечить гибкость, масштабируемость и автоматизацию проекта, что было важно при его разработке. Мы также вывели системные требования, необходимые для поддержания качественной и удобной работы приложения.

СПИСОК ЛИТЕРАТУРЫ

1. Погружаемся в Docker: Dockerfile и коммуникация между контейнерами // Хабр URL: <https://habr.com/ru/companies/infobox/articles/240623/>
2. Просто о микросервисах // Хабр URL: <https://habr.com/ru/companies/raiffeisenbank/articles/346380/?ysclid=mqvkskyxoe205661363>
3. Как работает Docker: подробный гайд от техлида // Skillbox URL: <https://skillbox.ru/media/code/kak-rabotaet-docker-podrobnyy-gayd-ot-tekhlista/>
4. Микросервисная архитектура // Википедия URL: https://ru.wikipedia.org/wiki/%D0%9C%D0%B8%D0%BA%D1%80%D0%BE%D1%81%D0%B5%D1%80%D0%B2%D0%B8%D1%81%D0%BD%D0%B0%D1%8F_%D0%B0%D1%80%D1%85%D0%B8%D1%82%D0%B5%D0%BA%D1%82%D1%83%D1%80%D0%B0?ysclid=mqyv8svrbq636829881
5. Документация Docker // Docker Docs URL: <https://docs.docker.com/>